Primo laboratorio

È importante saper distinguere tra chiacchere, pseudocodice e codice.
Una delle cose più importanti è dare **nomi sensati e adeguati** alle variabili, funzioni etc e commentare il più possibile, soprattutto quando non è facile capire il “ruolo” delle cose semplicemente dando uno sguardo. Una tecnica funzionante è quella di non scrivere codice pensando solo che quel codice dovrà essere capito dalla macchina, ma scriverlo come se lo stessimo spiegando a un nostro collega.

Non dobbiamo interessarci alle ottimizzazioni a **basso livello**.

Useremo la libreria *fstream* che comprende in e *out*, i quali permettono di riconoscere i tipi delle variabili passate ed ignorano spazi ed invii

Varie strutture dati in C++

Vettori:

CODING: VECTOR

Equivalente all’arraylist di java.

```
#include <vector>
//Crea vector di interi
vector<int> intvec;
//Crea vector di 7 float inizializzati a 0.5
vector<float> floatvec(7,0.5);
//Accedi agli elementi
floatvec[2] = floatvec[5] + 0.1;
//Aggiungi un elemento in fondo al vector
intvec.push_back(231);
//Cicla sugli elementi:
for(int i=0; i < intvec.size(); i++)
  intvec[i] = 12;
//Ridimensiona vector
intvec.resize(100);
```

Coppia di elementi:

CODING: PAIR

Coppia di elementi.

```
#include <utility>
//pair di intero e float
pair<int,float> coppia1;
//assegnazione elementi
coppia1.first = 2;
coppia1.second = 3.4;
coppia1 = make_pair(15,0.4);
//coppia di coppie
pair< pair<int,int>, pair<int,int> > c;
```

Ordinamento di elementi (sort):

CODING: SORT

```
#include <algorithm>
//ordinare un array di N elementi
sort(arr, arr + N);
//ordinare un vector
sort(vec.begin(), vec.end());
```

Coda:

CODING: CODA

```
#include <queue>
//Dichiarare coda di interi
queue<int> q;
//Aggiungere un elemento alla coda
q.push(23);
//Leggere l'elemento in testa alla coda
int el = q.front();
//Eliminare l'elemento in testa alla coda
q.pop();
//Controllare se la coda e vuota
if(q.empty())
   ...
```

Pila:

CODING: PILA

```
#include <stack>
//Dichiarare pila di interi
stack<int> s;
//Aggiungere un elemento in cima alla pila
s.push(23);
//Leggere l'elemento in cima alla pila
int el = s.top();
//Eliminare l'elemento in cima alla pila
s.pop();
//Controllare se la pila e vuota
if(s.empty())
   ...
```

Esiste il foreach, che itera su tutti gli elementi di un array, poi esiste il tipo *auto*, il quale prende qualsiasi tipo di variabile:

NOTE SU C++11 (I)

- For-each
- auto

```
vector<int> arr = ...;
for(int el:arr){
  cout << el << endl;
}
for(int& el:arr){
  el++;
}
auto d = 23;
for(auto& el:arr){
  el += d;
}
return arr;
```

Esiste anche il move operator, per definire funzioni che non dipendono dal tipo della variabile:

NOTE SU C++11 (II)

- Move operator

```
template<class T> void swap(T& a, T& b) {
  T tmp { std::move(a) };
  a = std::move(b);
  b = std::move(tmp);
}
```